

Titulación:	DAM	Curso:	1º
Módulo:	Programación	Fecha:	30-04-2026
Examen:	Tercer Trimestre	Duración:	
Nombre:		Calificación:	
Apellidos:			

RAs: RA7. Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.
RA9. Gestiona información almacenada en bases de datos manteniendo la integridad y consistencia de los datos.

NORMAS E INSTRUCCIONES

- Como regla general, cada error u omisión en un ejercicio respecto a lo que se pide o se ve claramente restará parte del valor de la nota total del mismo.
- A la hora de calificar los ejercicios puntuará no solo la correcta ejecución sino también la calidad de la solución aportada.
- La rúbrica usada para la corrección se publicará junto a los resultados del examen. En caso de errores muy o poco graves no contemplados en la rúbrica se valorará de forma objetiva.
- Un ejercicio que no llegue a ejecutarse porque de un error o falle ante entradas como las descritas en el enunciado, en los ejemplos que aparecen en el mismo o en la descripción del enunciado no puntuará mas de un 2 sobre 10
- No se valorará con mas de un 2 sobre 10 ningún ejercicio que no vaya claramente encaminado a resolver exactamente lo que se pide en el enunciado
- La nota de un ejercicio no puede ser negativa.

MUY IMPORTANTE: Crea un package que se llame examen2 y crea en él todas las clases que precisas para ambos ejercicios. Entrega al final del examen todos los archivos .java que hayas creado en ese package. No entregues nada que no sea esto o que no tenga que ver con la resolución del ejercicio.

1. (R9) Vamos a trabajar con la base de datos classicmodels que ya conoces

Crea una función en Java que reciba como argumento el nombre de una ciudad y liste todos los empleados de esa ciudad y su email. Por ejemplo, si tu función recibe como argumento Paris listará lo siguiente:

Hay 5 empleados en la oficina de Paris. Sus datos son:

Bondur, Gerard (gbondur@classicmodelcars.com)

Bondur, Loui (lbondur@classicmodelcars.com)

Hernandez, Gerard (ghernande@classicmodelcars.com)

Castillo, Pamela (pcastillo@classicmodelcars.com)

Gerard, Martin (mgerard@classicmodelcars.com)

Contempla en tu función que la ciudad no tenga oficina o no existan empleados en ella:

No existe oficina en Madrid

o

La oficina de Río de Janeiro no tiene empleados

NOTA: Los datos de la oficina de París que aparecen antes son correctos, la oficina de Madrid efectivamente no existe y todas las oficinas de Sakila tienen algún empleado pero eso lo solucionamos en el siguiente ejercicio ;-)

2. (R9) Crea una función en Java que reciba dos nombres de ciudades y mueva todos los empleados de la oficina de una a la otra. Por ejemplo, si tu función recibe la orden de desplazar los empleados de Tokyo a Paris debería de dar la siguiente salida por pantalla:

**Moviendo todos los empleados de Tokyo a Paris.
Se van a mover 2 empleados de Tokyo a París
La oficina de París tiene ahora 7 empleados**

NOTA: En la oficina de Tokyo aparecen originalmente 2 empleados mientras que en la de París son 5. Ahora ya puedes probar la parte del ejercicio anterior donde se introduce una oficina sin empleados ;-)

3. (R7). Queremos crear una clase para que sirva como base para un juego de cartas coleccionables de temática de fantasía. La clase se llamará Carta y los atributos de la misma son cinco: nombre, tipo, coste, descripción y ubicación. Por ejemplo:

Nombre: Black Lotus

Tipo: Artefacto

Coste: 0

Descripción: Sacrifica el Black Lotus. Agrega 3 manás de cualquier color a tu pool de maná

Ubicación: Biblioteca

Crea el constructor de la clase sabiendo que todos los campos son texto salvo el coste que es un número entero. La ubicación inicial es siempre Biblioteca, así que ese dato no se lo facilitamos al constructor

Sobreescribe los métodos necesarios para

- que la salida impresa de un objeto sea como la anterior
- que al ordenar diversos objetos del tipo Carta en un ArrayList la ordenación se realice alfabéticamente por el nombre de la misma

Crea al menos tres objetos con diferentes costes, mételos en un arraylist, ordénalo y muestra el resultado por consola para demostrar que el funcionamiento es correcto

Define, por último, una interfaz que se llame Ubicacion y asígnala a tu clase. La interfaz tendrá tres métodos llamados cementerio, biblioteca y mano. Cada uno de ellos cambiará la ubicación de la carta a una zona de juego diferente. Por ejemplo, si tienes un objeto de tipo Carta que se llama carta5 y llamas al método carta5.cementerio() la ubicación de tu carta pasará a ser Cementerio. Igual con los otros dos métodos salvo que cambiaran la ubicación a Biblioteca y Mano respectivamente

NOTA: No hace falta que sepas de Magic. Pon los textos y costes que te de la gana en los campos. No obstante, si lo necesitas, aquí te pongo un par de cartas mas:

Nombre: Luna de Sangre

Tipo: Encantamiento

Coste: 3

Descripción: Todas las tierras no básicas son Montañas

Nombre: Ave del Paraíso

Tipo: Criatura

Coste: 1

Descripción: Vuela. Agrega un maná de cualquier color

4. (R7). Crea dos funciones lambda que sirvan, respectivamente, para calcular el perímetro y el área de un rectángulo. Las funciones, ambas, deberían de recibir dos números (con decimales) correspondientes a los dos lados del rectángulo y devolver asimismo un número redondeado a tres decimales correspondiente al cálculo que se pide.

Prueba que ambas funcionan con un par de llamadas de ejecución como ejemplo